```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
```

```
df = pd.read_csv('/content/titanic.csv')
df.head()
```

	Passengerid	Age	Fare	Sex	sibsp	zero	zero.1	zero.2	zero.3	zero.4	.
0	1	22.0	7.2500	0	1	0	0	0	0	0	
1	2	38.0	71.2833	1	1	0	0	0	0	0	
2	3	26.0	7.9250	1	0	0	0	0	0	0	
3	4	35.0	53.1000	1	1	0	0	0	0	0	
4	5	35.0	8.0500	0	0	0	0	0	0	0	

5 rows × 28 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1309 entries, 0 to 1308
Data columns (total 28 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Passengerid  1309 non-null   int64
 1   Age          1309 non-null   float64
 2   Fare         1309 non-null   float64
 3   Sex          1309 non-null   int64
 4   sibsp        1309 non-null   int64
 5   zero         1309 non-null   int64
 6   zero.1       1309 non-null   int64
 7   zero.2       1309 non-null   int64
 8   zero.3       1309 non-null   int64
 9   zero.4       1309 non-null   int64
 10  zero.5       1309 non-null   int64
 11  zero.6       1309 non-null   int64
 12  Parch        1309 non-null   int64
 13  zero.7       1309 non-null   int64
 14  zero.8       1309 non-null   int64
 15  zero.9       1309 non-null   int64
 16  zero.10      1309 non-null   int64
 17  zero.11      1309 non-null   int64
 18  zero.12      1309 non-null   int64
```

```
 19  zero.13      1309 non-null   int64
 20  zero.14      1309 non-null   int64
 21  Pclass       1309 non-null   int64
 22  zero.15      1309 non-null   int64
 23  zero.16      1309 non-null   int64
 24  Embarked     1307 non-null   float64
 25  zero.17      1309 non-null   int64
 26  zero.18      1309 non-null   int64
 27  2urvived     1309 non-null   int64
dtypes: float64(3), int64(25)
memory usage: 286.5 KB
```

```
df.describe()
```

Passengerid	Age	Fare	Sex	sibsp	zero	zer
1309.000000	1309.000000	1309.000000	1309.000000	1309.000000	1309.0	13
655.000000	29.503186	33.281086	0.355997	0.498854	0.0	
378.020061	12.905241	51.741500	0.478997	1.041658	0.0	
1.000000	0.170000	0.000000	0.000000	0.000000	0.0	
328.000000	22.000000	7.895800	0.000000	0.000000	0.0	
655.000000	28.000000	14.454200	0.000000	0.000000	0.0	
982.000000	35.000000	31.275000	1.000000	1.000000	0.0	
1309.000000	80.000000	512.329200	1.000000	8.000000	0.0	

28 columns

```
df.isnull().sum()
```

	0
Passengerid	0
Age	0
Fare	0
Sex	0
sibsp	0
zero	0
zero.1	0
zero.2	0
zero.3	0
zero.4	0
zero.5	0
zero.6	0
Parch	0
zero.7	0
zero.8	0
zero.9	0
zero.10	0
zero.11	0
zero.12	0
zero.13	0
zero.14	0
Pclass	0
zero.15	0
zero.16	0
Embarked	2
zero.17	0
zero.18	0
2urvived	0

dtype: int64

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

```
/tmp/ipykernel_15568/1933487976.py:1: FutureWarning: A value is trying to be set
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Age'].fillna(df['Age'].median(), inplace=True)
```

```
df.columns
```

```
Index(['Passengerid', 'Age', 'Fare', 'Sex', 'sibsp', 'zero', 'zero.1',
       'zero.2', 'zero.3', 'zero.4', 'zero.5', 'zero.6', 'Parch', 'zero.7',
       'zero.8', 'zero.9', 'zero.10', 'zero.11', 'zero.12', 'zero.13',
       'zero.14', 'Pclass', 'zero.15', 'zero.16', 'Embarked', 'zero.17',
       'zero.18', '2urvived'],
      dtype='object')
```

```
df.columns = df.columns.str.lower()
```

```
df.rename(columns={'2urvived': 'survived'}, inplace=True)
```

```
df = df.loc[:, ~df.columns.str.contains('zero')]
```

```
print(df.columns)
```

```
Index(['passengerid', 'age', 'fare', 'sex', 'sibsp', 'parch', 'pclass',
       'embarked', 'survived'],
      dtype='object')
```

SKEWNESS

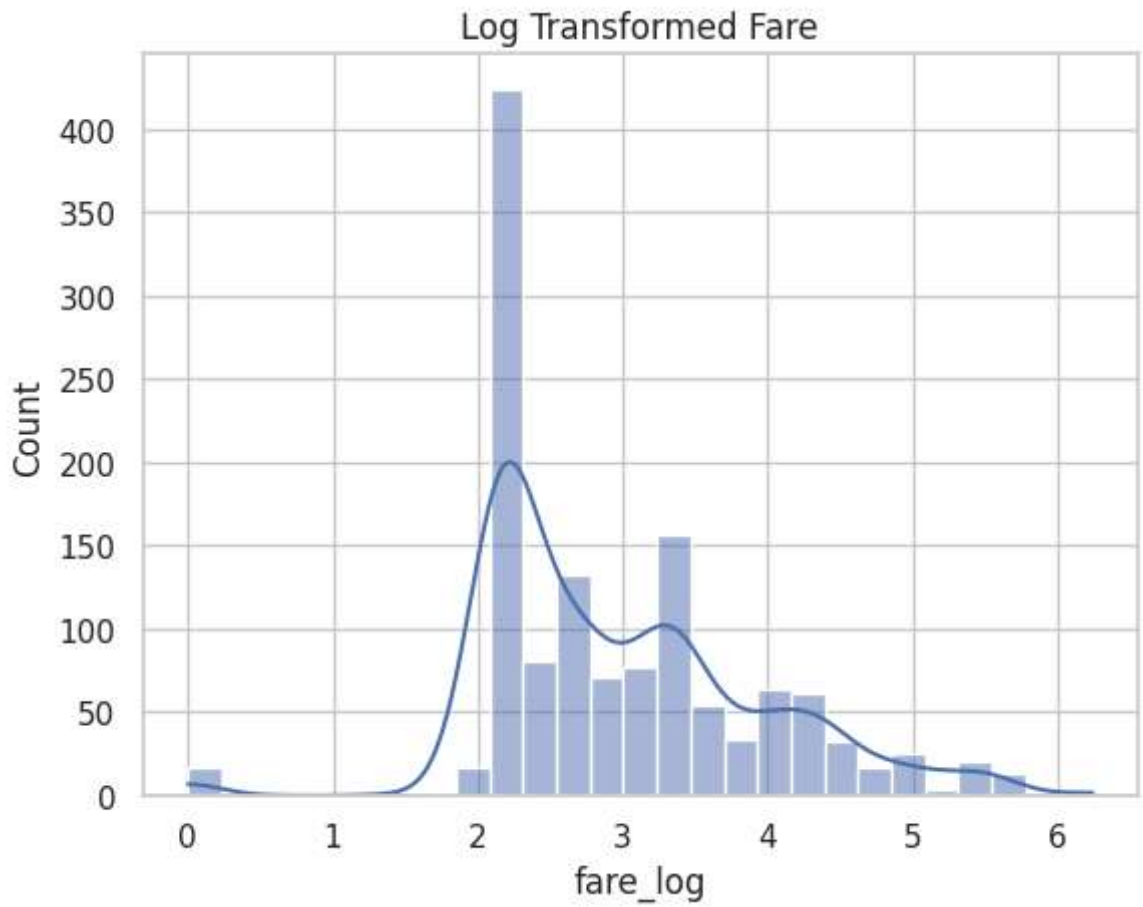
```
print(df[['age', 'fare']].skew())
```

```
age     0.540987
fare    4.369510
dtype: float64
```

- Fare is highly positively skewed.
- Age has slight skewness.

```
df['fare_log'] = np.log1p(df['fare'])

sns.histplot(df['fare_log'], kde=True)
```

```
plt.title("Log Transformed Fare")
plt.show()
```



- Log transformation reduces skewness in fare distribution.

Multicollinearity

```
corr = df.corr(numeric_only=True)
print(corr)
```

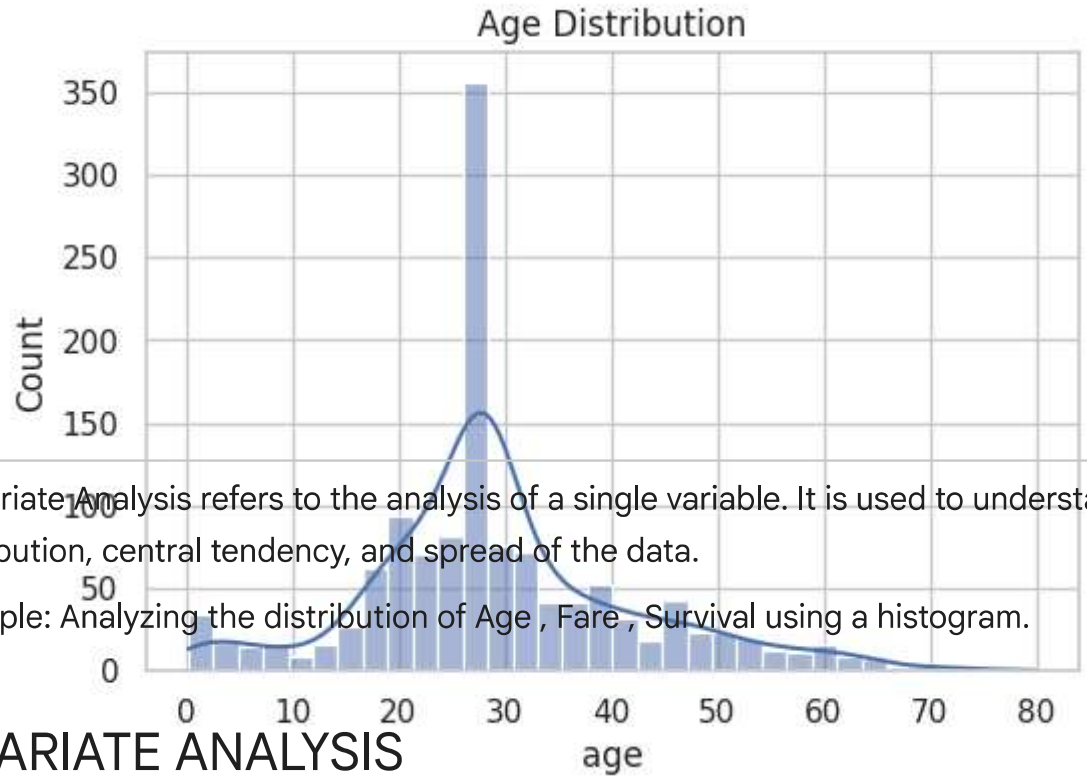
```
             passengerid       age      fare       sex     sibsp     parch  \
passengerid     1.000000  0.025799  0.031128 -0.013406 -0.055224  0.008942
age             0.025799  1.000000  0.178182 -0.053663 -0.189972 -0.125851
fare            0.031128  0.178182  1.000000  0.185681  0.160349  0.221635
sex            -0.013406 -0.053663  0.185681  1.000000  0.109609  0.213125
sibsp          -0.055224 -0.189972  0.160349  0.109609  1.000000  0.373587
parch           0.008942 -0.125851  0.221635  0.213125  0.373587  1.000000
pclass         -0.038354 -0.377908 -0.558683 -0.124617  0.060832  0.018322
embarked       -0.051977 -0.063424 -0.238131 -0.097960  0.065567  0.044772
survived       -0.331493 -0.055862  0.173786  0.404020 -0.014375  0.054908
fare_log        0.019513  0.183750  0.805479  0.250400  0.311667  0.323565

               pclass  embarked  survived  fare_log
passengerid -0.038354 -0.051977 -0.331493  0.019513
```

```
age         -0.377908 -0.063424 -0.055862  0.183750
fare        -0.558683 -0.238131  0.173786  0.805479
sex         -0.124617 -0.097960  0.404020  0.250400
sibsp        0.060832  0.065567 -0.014375  0.311667
parch        0.018322  0.044772  0.054908  0.323565
pclass       1.000000  0.185479 -0.244686 -0.685933
embarked     0.185479  1.000000 -0.091394 -0.212124
survived    -0.244686 -0.091394  1.000000  0.238255
fare_log    -0.685933 -0.212124  0.238255  1.000000
```

- No very high correlations (>0.8), so multicollinearity is low.
- Pclass and Fare show moderate correlation.

UNIVARIATE Analysis

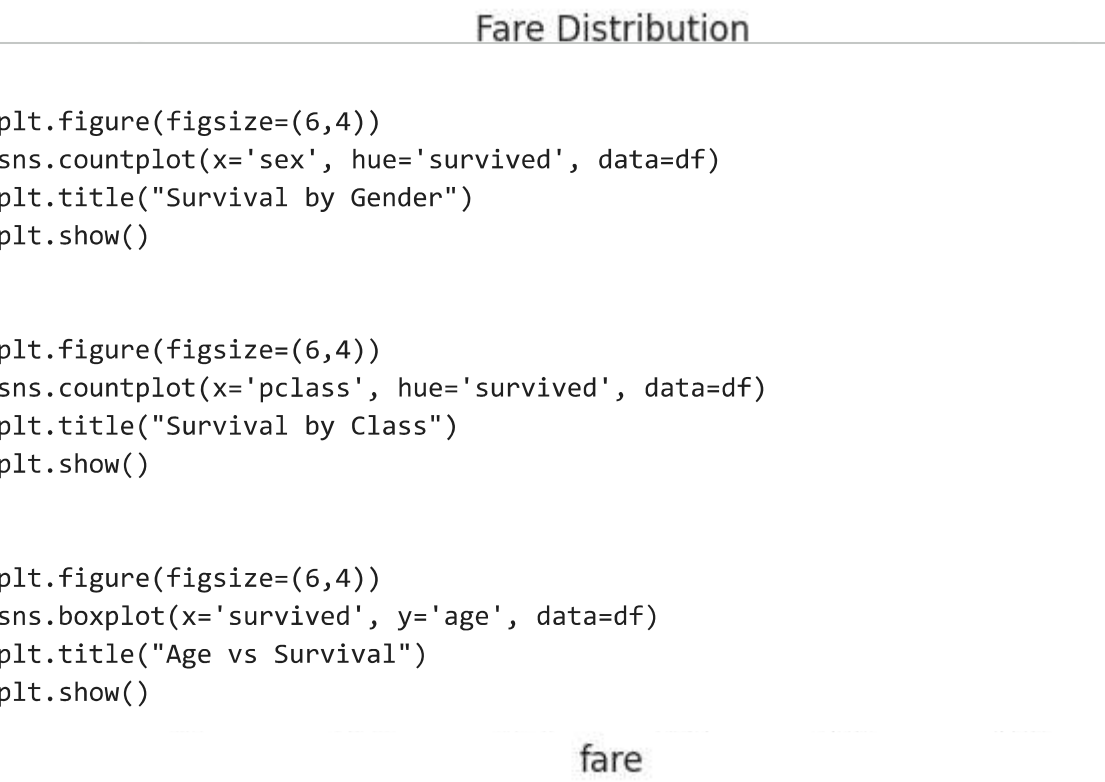
```
plt.figure(figsize=(6,4))
sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
plt.show()


plt.figure(figsize=(6,4))
sns.histplot(df['fare'], kde=True)
plt.title("Fare Distribution")
plt.show()


plt.figure(figsize=(6,4))
sns.countplot(x='survived', data=df)
plt.title("Survival Count")
plt.show()
```



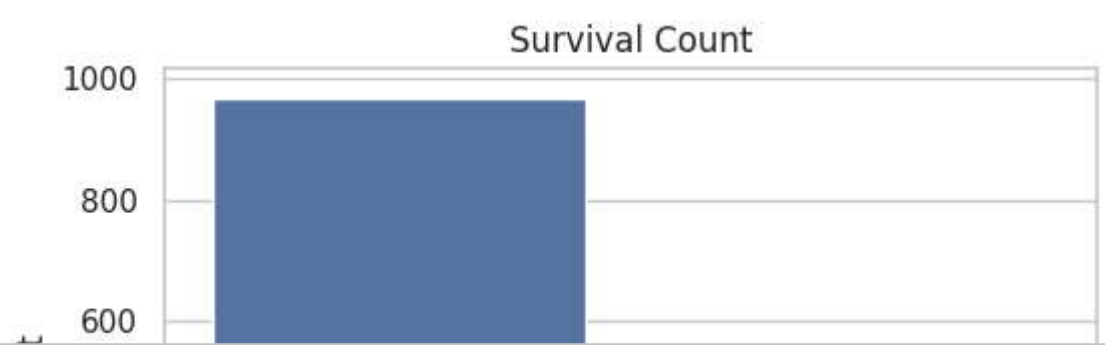
Univariate Analysis refers to the analysis of a single variable. It is used to understand the distribution, central tendency, and spread of the data.

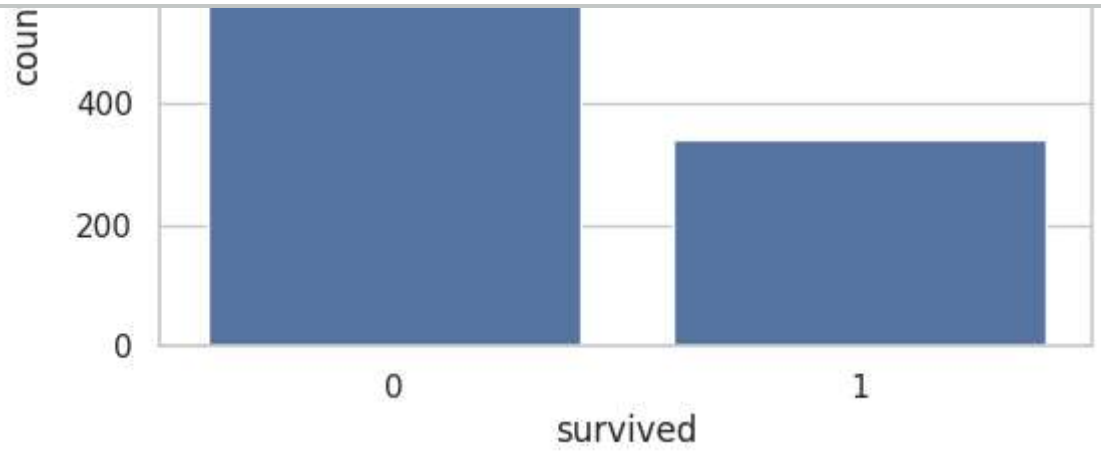
Example: Analyzing the distribution of Age , Fare , Survival using a histogram.

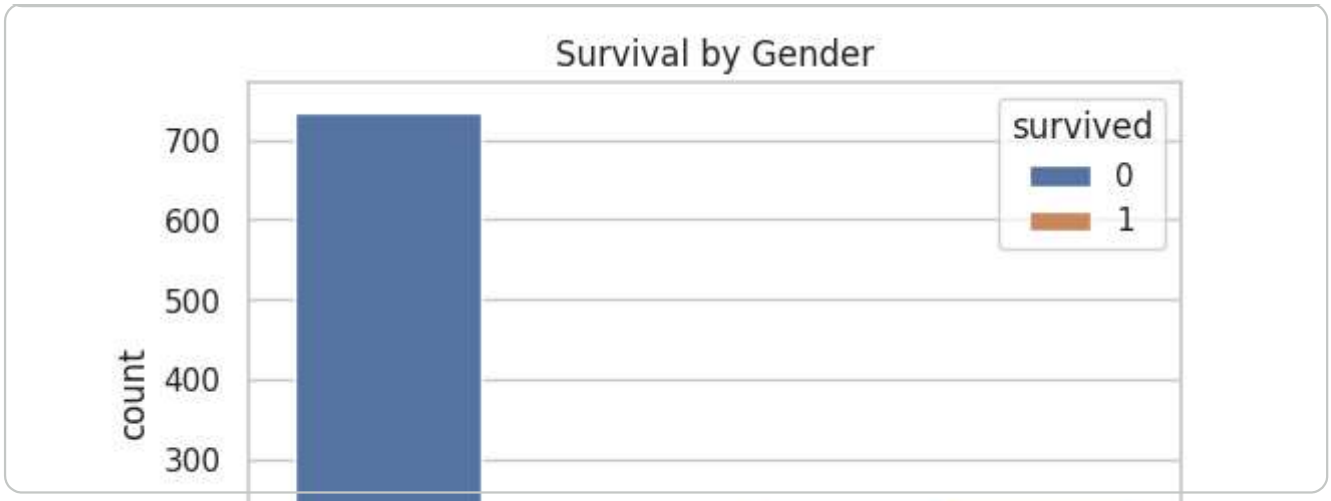
BIVARIATE ANALYSIS




```
plt.figure(figsize=(6,4))
sns.countplot(x='sex', hue='survived', data=df)
plt.title("Survival by Gender")
plt.show()


plt.figure(figsize=(6,4))
sns.countplot(x='pclass', hue='survived', data=df)
plt.title("Survival by Class")
plt.show()


plt.figure(figsize=(6,4))
sns.boxplot(x='survived', y='age', data=df)
plt.title("Age vs Survival")
plt.show()
```



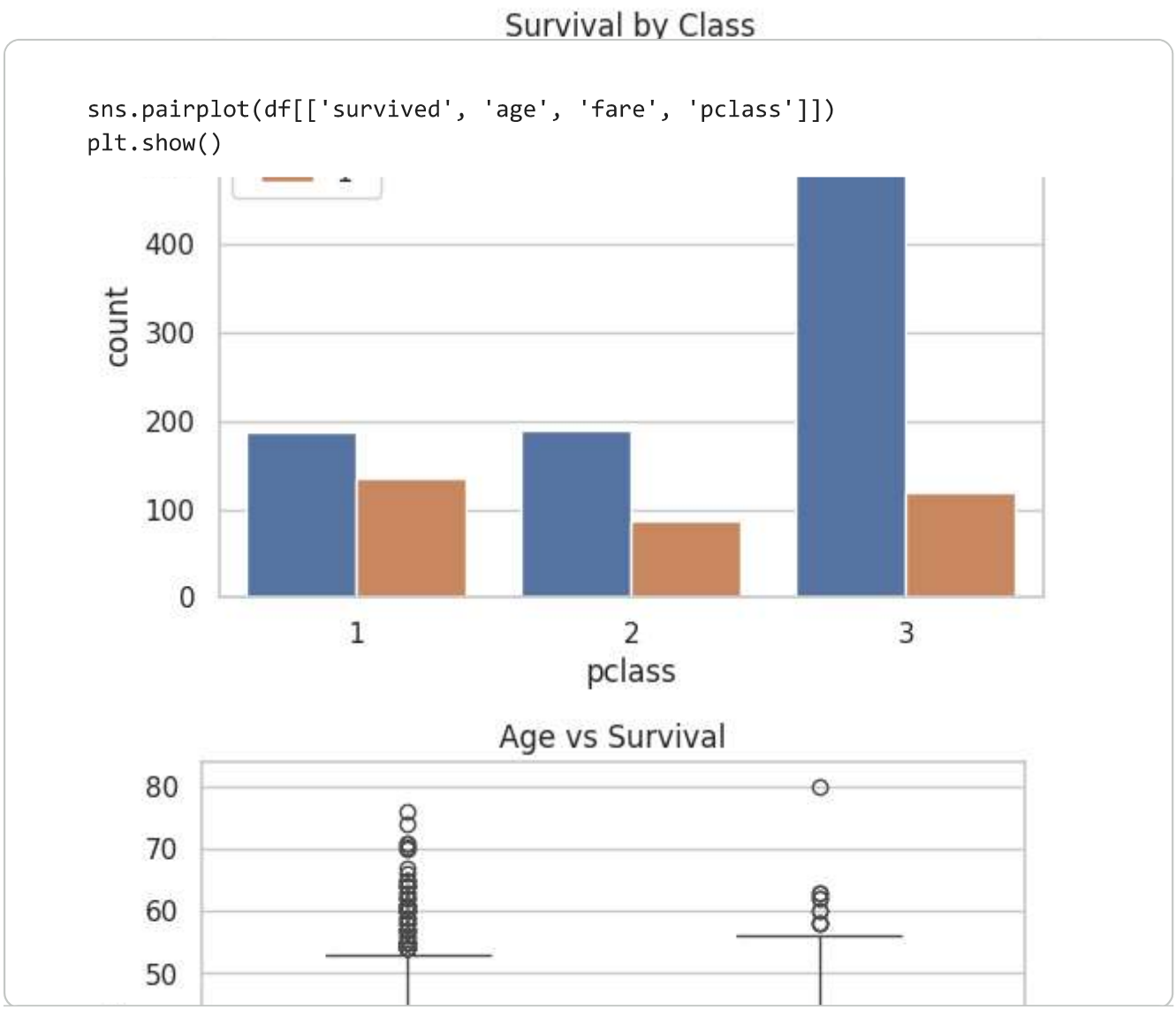

coun
400
200
0
0
1
survived



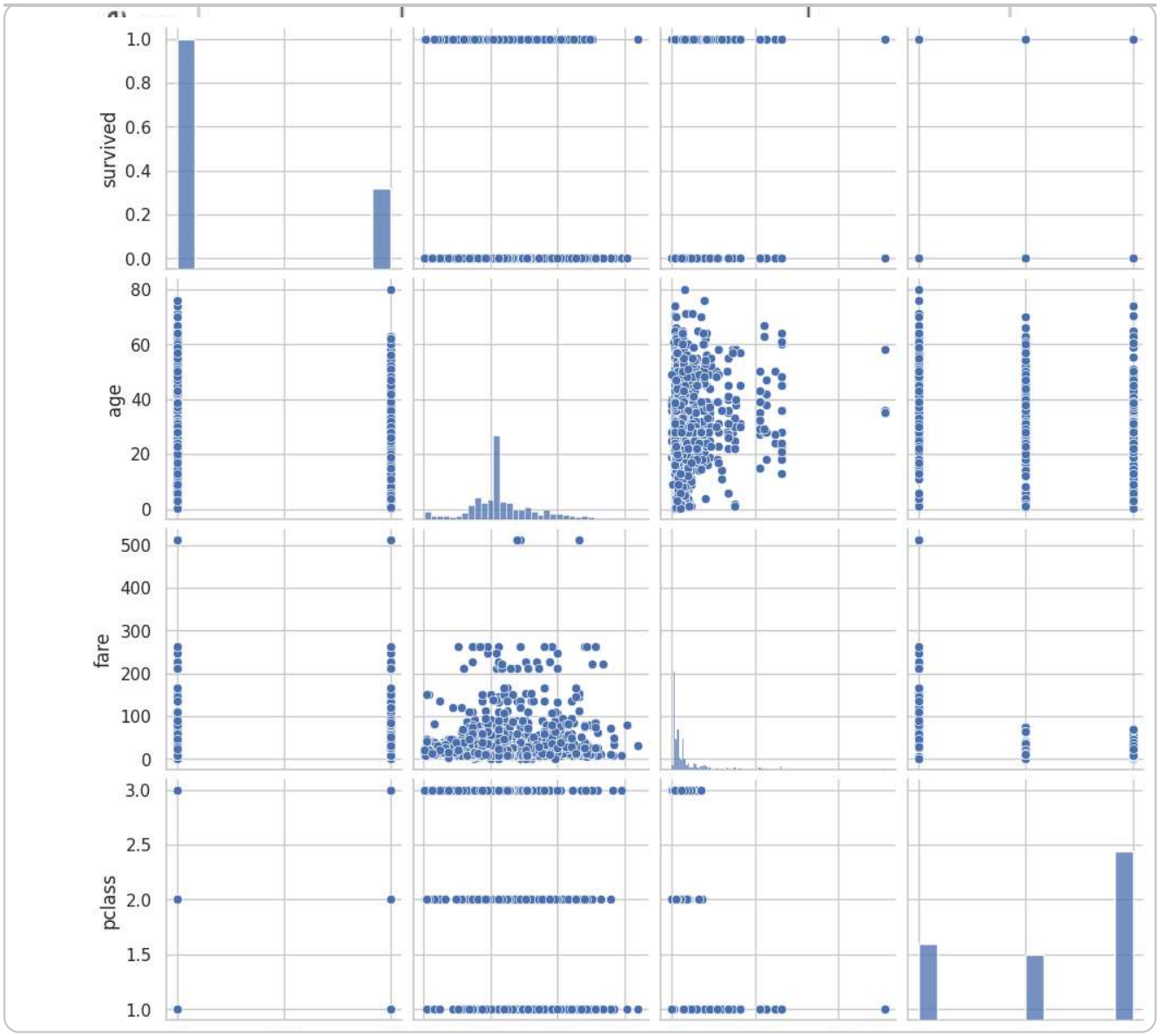
Bivariate Analysis involves the analysis of two variables to understand the relationship between them.

Example: Comparing Survival with Gender to see how gender affects survival rates.

MULTIVARIATE ANALYSIS




```
sns.pairplot(df[['survived', 'age', 'fare', 'pclass']])
plt.show()
```

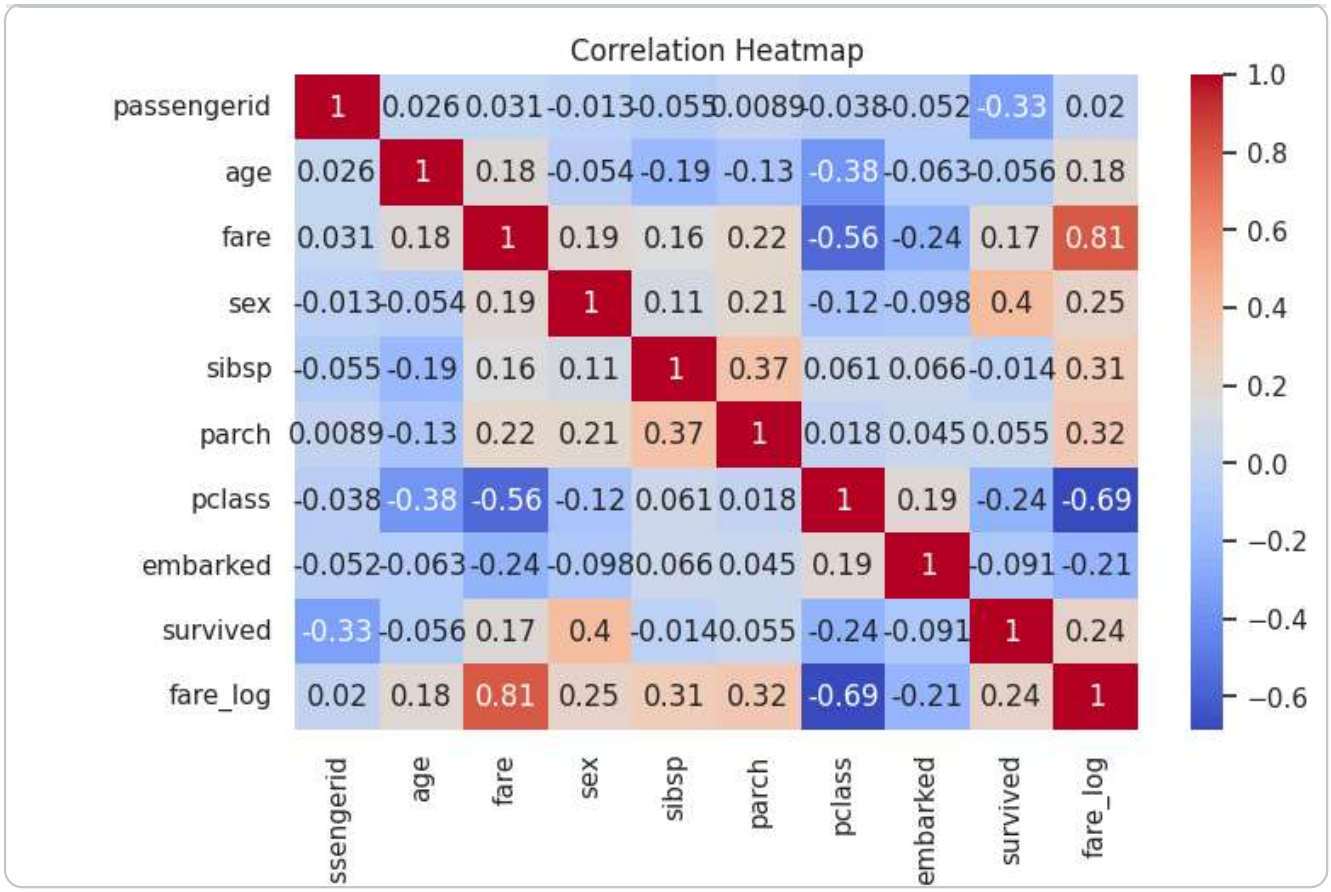


Multivariate Analysis involves analyzing more than two variables simultaneously to understand complex relationships.

Example: Using a pairplot to analyze the relationship between Age, Fare, Passenger Class, and Survival.

CORRELATION HEATMAP

```
plt.figure(figsize=(8,5))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



VALUE COUNTS

```
print(df['survived'].value_counts())
print(df['sex'].value_counts())
print(df['pclass'].value_counts())
```

```
survived
0    967
1    342
Name: count, dtype: int64
sex
0    843
1    466
Name: count, dtype: int64
pclass
3    709
1    323
2    277
Name: count, dtype: int64
```

SKEWNESS

```
print(df[['age', 'fare']].skew())
```